

LAPORAN PRAKTIKUM
SISTEM OPERASI

MODUL XI
MEMORI XINU



Universitas
Telkom

Disusun Oleh:

GHAZA ZIDANE NURRAIHAN

2311104038

SE-07-01

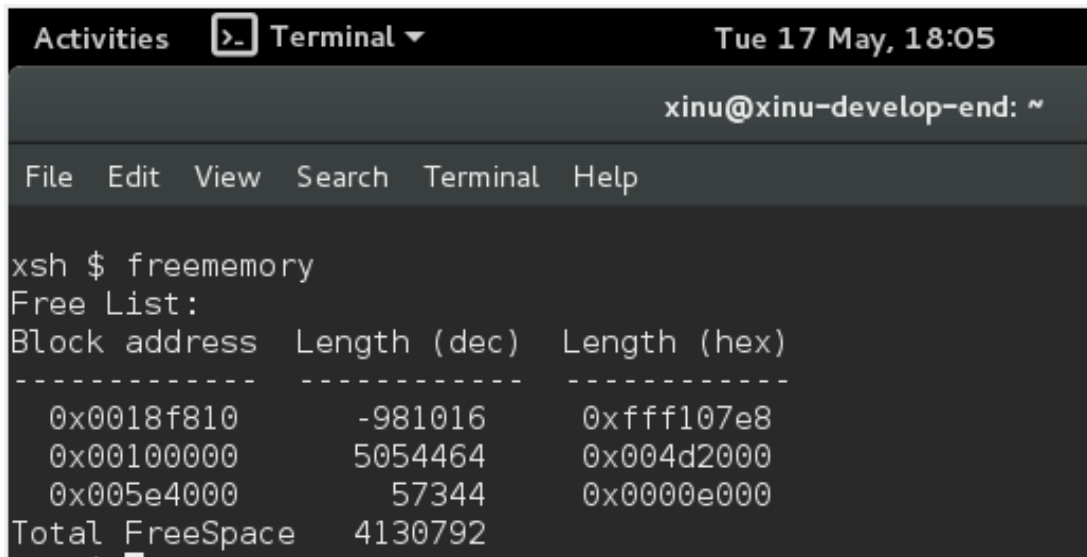
PROGRAM STUDI S1 REKAYASA PERANGKAT LUNAK
DIREKTORAT KAMPUS PURWOKERTO
UNIVERSITAS TELKOM

2025

Jurnal

1. [80 poin] Buatlah perintah baru bernama **freememory** yang memiliki dua fungsi berikut:
 - a. [40 poin] Menampilkan seluruh free memory block yang tercatat dalam free memory list pada Xinu.
 - b. [40 poin] Menghitung dan menampilkan total ukuran free memory berdasarkan seluruh block yang ada pada list tersebut.

Output yang di inginkan:

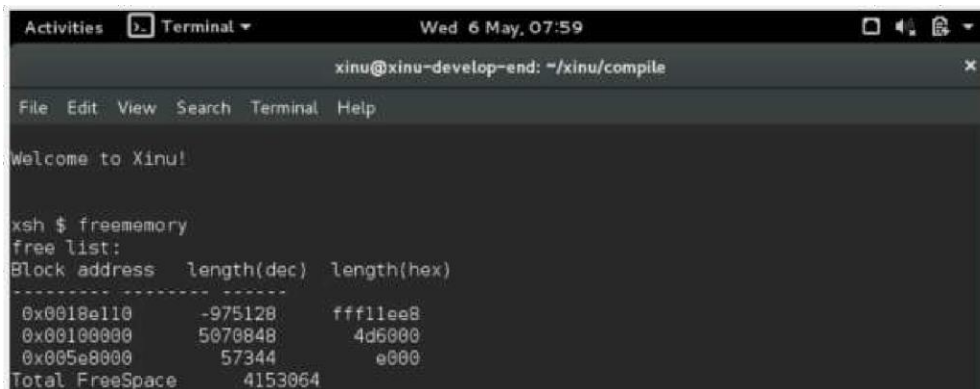


```
Activities Terminal Tue 17 May, 18:05
xinu@xinu-develop-end: ~

File Edit View Search Terminal Help

xsh $ freememory
Free List:
Block address  Length (dec)  Length (hex)
-----
0x0018f810    -981016    0xffff107e8
0x00100000     5054464    0x004d2000
0x005e4000      57344     0x0000e000
Total FreeSpace 4130792
```

Jawab :



```
Activities Terminal Wed 6 May, 07:59
xinu@xinu-develop-end: ~/xinu/compile

File Edit View Search Terminal Help

Welcome to Xinu!

xsh $ freememory
free list:
Block address  length(dec)  length(hex)
-----
0x0018e110    -975128    fff11ee8
0x00100000     5070848    4d6000
0x005e8000      57344      e000
Total FreeSpace 4153064
```

2. [4 poin per subsoal] Jawablah pertanyaan berikut:
 - a. Mengapa Xinu memisahkan data segment dan BSS segment?
 - b. Bagaimana alokasi dan dealokasi memori selama eksekusi memengaruhi ukuran free space?
 - c. Mengapa penggunaan heap lebih berpotensi menimbulkan masalah dibandingkan stack?
 - d. Mengapa Xinu menggunakan struktur linked list untuk menyimpan free block?

e. Apa tantangan utama dalam penggunaan heap di Xinu?

Jawab:

- a. Xinu memisahkan Data Segment dan BSS Segment untuk meningkatkan efisiensi pengelolaan memori. Data Segment digunakan untuk menyimpan variable global dan statis yang telah memiliki nilai awal (initialized data), sedangkan BSS Segment digunakan untuk variable global dan statis yang belum diinisialisasi atau bernilai nol secara default. Dengan pemisahan ini, ruang penyimpanan program dapat dihemat karena data pada BSS tidak perlu disimpan secara eksplisit di file executable dan cukup dialokasikan saat program dijalankan.
- b. Ketika memori di alokasikan, sebagai ruang kosong (free space) akan digunakan sehingga ukuran free space berkurang. Sebaliknya, saat memori yang tidak lagi diperlukan didealokasikan atau dikembalikan ke sistem, ukuran free space akan bertambah kembali. Namun, proses alokasi dan dealokasi yang terjadi berulang kali dapat menyebabkan fragmentasi memori, sehingga meskipun total free space masih besar, ruang kosong tersebut mungkin tersebar dalam blok-blok kecil yang sulit dimanfaatkan untuk kebutuhan alokasi yang lebih besar.
- c. Heap lebih rentan menimbulkan masalah karena pengelolaan memorinya dilakukan secara dinamis selama program berjalan. Kesalahan seperti memory leak, dangling pointer, dan fragmentasi memori sering terjadi apabila alokasi dan dealokasi tidak dilakukan dengan benar. Berbeda dengan stack yang menggunakan mekanisme Last In First Out (LIFO) sehingga pengalokasian dan pembebasan memori berlangsung secara otomatis dan terstruktur, heap memerlukan pengelolaan yang lebih kompleks sehingga Risiko kesalahan lebih tinggi.
- d. Xinu menggunakan linked list untuk menyimpan free block karena struktur ini fleksibel dan efisien dalam mengelola ruang memori yang ukurannya bervariasi. Linked list memudahkan sistem untuk menambah, menghapus, membagi, maupun menggabungkan blok memori yang kosong tanpa harus memindahkan data lainnya. Selain itu, penggunaan linked list memungkinkan proses pencarian dan pengelolaan blok bebas dilakukan dengan overhead yang relatif kecil, sehingga cocok untuk kebutuhan manajemen memori pada sistem operasi seperti Xinu.
- e. Tantangan utama dalam penggunaan heap di OS Xinu berakar dari strukturnya yang sangat minimalis dan berbasis alamat fisik langsung (physical addressing) tanpa adanya proteksi Memory Management Unit (MMU). Hal ini memicu terjadinya fragmentasi eksternal yang masif karena pengelolaan memori menggunakan free list sederhana yang rentan terpecah menjadi blok-blok kecil, mengakibatkan kegagalan alokasi data besar meskipun total sisa memori sebenarnya mencukupi. Ketidadaan isolasi memori virtual ini juga menciptakan risiko keamanan tinggi, di mana pointer liar (dangling pointer) atau buffer overflow dari satu proses dapat langsung merusak (corrupt) data milik proses lain atau bahkan merusak struktur data inti kernel itu sendiri. Masalah ini diperparah oleh sistem pelacakan Xinu yang tidak mencatat kepemilikan blok memori, sehingga setiap kelalaian dalam membebaskan memori akan menyebabkan kebocoran memori (memory leak) permanen yang hanya bisa dipulihkan lewat reboot total mesin backend. Terakhir, karena free list merupakan

shared resource global, setiap proses alokasi (getmem) dan dealokasi (freemem) terpaksa harus mematikan interupsi sistem (disable interrupts) demi menjaga konsistensi data, yang pada gilirannya menimbulkan overhead sinkronisasi dan berpotensi mengacaukan performa ketepatan waktu pada sistem real-time Xinu.

REFERENSI

<https://en.wikipedia.org/wiki/Xinu>